GitHub Actions – Complete Practical Guide

This guide is designed for engineers who already understand CI/CD concepts but want real hands■on mastery of GitHub Actions. It walks step■by■step from basics to production■level pipelines including Docker, Kubernetes, Terraform, AWS, and real DevOps scenarios.

1. What is GitHub Actions?

GitHub Actions is a CI/CD and automation platform built into GitHub. It allows you to automate build, test, deploy, security scanning, and infrastructure tasks directly from your repository.

Core Components:

1. Workflow – YAML file that defines automation
2. Event – Trigger like push, PR, schedule
3. Job – Set of steps running on a runner
4. Step – Individual task like run command
5. Runner – Machine executing the workflow

Why DevOps engineers love it: native GitHub integration, powerful marketplace, matrix builds, reusable workflows.

2. First Workflow Hands■On

Create file: .github/workflows/ci.yml

```
name: First Workflow
on: push

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run script
        run: echo "Hello DevOps"
```

Push code → See Actions tab → Watch logs.

3. Real Java Build (Maven/Gradle)

For your DevOps background and Java container projects, you should automate Maven/Gradle builds.

```
name: Java Build
on: push

jobs:
 build:
   runs-on: ubuntu-latest
   steps:
    - uses: actions/checkout@v4
    - name: Setup Java
      uses: actions/setup-java@v4
      with:
        distribution: temurin
        java-version: 17
    - run: mvn clean package
```

Add test reports, caching dependencies, uploading artifacts.

4. Docker Build and Push

Important for your Kubernetes and EKS experience.

```
      - name: Login to DockerHub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKER_USER }}
          password: ${{ secrets.DOCKER_PASS }}

      - name: Build Image
        run: docker build -t myapp:${{ github.sha }} .

      - name: Push Image
        run: docker push myapp:${{ github.sha }}
```

Use GitHub Container Registry or ECR.

5. Deploy to AWS EKS using Terraform

Matches your AWS + Terraform learning plan.

```
- name: Configure AWS
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123:role/github-role
    aws-region: ap-south-1

- name: Terraform Apply
  run: |
    terraform init
    terraform apply -auto-approve
```

Combine with IRSA knowledge you are learning.

6. Advanced Concepts

Must■know topics for professional DevOps engineers:

1. Matrix builds
2. Reusable workflows
3. Self■hosted runners
4. Environment protection rules
5. Secrets management
6. Caching dependencies
7. Manual approvals

Learn to debug using step logs and 'act' CLI locally.

7. Production DevOps Projects to Build

Since you want strong resume projects, build these pipelines:

1. Java app → Docker → ECR → EKS deploy
2. Terraform infra → Plan on PR → Apply on merge
3. Security scan using Trivy/Snyk
4. Observability stack deploy (Prometheus/Grafana)
5. Multi■env deployment: dev → staging → prod

Each project should include README, architecture diagram, and pipeline screenshots.

8. Daily Practice Plan (2 Weeks)

Day 1■2: Basic workflows

Day 3■4: Java build + artifacts

Day 5■6: Docker pipeline

Day 7■8: AWS deploy

Day 9■10: Terraform CI/CD

Day 11■12: Matrix + reusable workflows

Day 13■14: Build full production pipeline.

9. Troubleshooting Cheatsheet

Common issues:

1. Permission denied → Check GITHUB_TOKEN scope
2. Docker push fails → Secrets misconfigured
3. AWS role fail → Wrong trust policy
4. Terraform stuck → Backend lock issue

Always check runner logs and enable debug logging.